Actor-Critic Methods

Abstract

This topic explores actor-critic methods in reinforcement learning, which integrate policy gradient and value function approaches to optimize decision-making in continuous and high-dimensional action spaces. It covers deterministic policy gradients, deep deterministic policy gradients, and advanced extensions like double-Q and Twin Delayed DDPG (TD3). These methods address the limitations of value-based approaches, offering improved stability and performance in complex environments. Students will gain a comprehensive understanding of actor-critic architectures, their implementation, and their practical applications in modern reinforcement learning.

Learning Objectives

- Understand and be able to implement the stochastic actor-critic
- Understand and be able to implement the actor-critic architecture and the DPG algorithm
- Understand and be able to implement deep deterministic policy gradients (DDPG)
- Explore TD3 and its extensions

Literature

This lecture draws from the following sources:

Primary Sources:

Sutton & Barto Textbook:

- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press. Online preprint (https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf)
  - Chapter 13: Policy Gradient Methods
  - Section 13.5: Actor-Critic Methods
  - Section 13.7: Policy Parameterization for Continuous Actions

Deterministic Policy Gradient (DPG):

- Silver, D., et al. (2014). "Deterministic Policy Gradient Algorithms." *ICML 2014*. PDF (http://proceedings.mlr.press/v32/silver14.pdf)

Deep Deterministic Policy Gradient (DDPG):

- Lillicrap, T. P., et al. (2015). "Continuous control with deep reinforcement learning." *ICLR 2016*. PDF (https://arxiv.org/pdf/1509.02971.pdf)

Twin Delayed DDPG (TD3):

- Fujimoto, S., et al. (2018). "Addressing Function Approximation Error in Actor-Critic Methods." *ICML 2018*. PDF (https://arxiv.org/pdf/1802.09477.pdf)

Modern Lecture Materials:

- Eysenbach, B. (2025). "Lecture 14: Actor Critic Methods." *COS435/ECE 433: Introduction to RL*. PDF (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-14-ddpg.pdf)
- Levine, S. (2023). "Lecture 9: Advanced Policy Gradients." *CS285: Deep RL*. PDF (https://rail.eecs.berkeley.edu/deeprlcourse-fa23/deeprlcourse-fa23/static/slides/lec-9.pdf)

Motivation & Gap Analysis

What We Know from Previous Lectures:

- **L1-L2:** Core RL concepts (MDPs, Bellman equations, policies, value functions)
- **L3:** Policy gradient methods (REINFORCE) for direct policy optimization
- **L4:** Dynamic programming for exact solutions in known environments
- **L5:** Monte Carlo methods for learning from complete episodes
- **L6:** Temporal-Difference learning (SARSA, Q-learning) for efficient online learning
- **L7:** Deep Q-Networks (DQN) for high-dimensional state spaces using function approximation

The Challenge We Face:

While DQN successfully addresses the curse of dimensionality in **state spaces**, it fundamentally fails in **continuous action spaces**:

1. **Discrete Action Limitation:** DQN requires computing $\max_{a'} Q(s', a')$, which involves:
    - Evaluating Q-values for all possible actions
    - Finding the action with maximum Q-value
    - This is only tractable for **discrete, finite action sets**
2. **Continuous Actions are Everywhere:** Real-world problems demand continuous control:
    - Robot joint angles (e.g., arm rotation: $[-180^\circ, +180^\circ]$)
    - Vehicle steering and throttle (e.g., steering: $[-1, +1]$, throttle: $[0, 1]$)
    - Chemical process control (e.g., temperature, pressure, flow rates)
    - Financial trading (e.g., portfolio weights, position sizes)

Why Policy Gradients Alone Are Not Enough:

From L3, we learned REINFORCE for policy optimization:

- **High variance:** Requires many episodes to estimate gradients accurately
- **Sample inefficiency:** Learns only from complete episode returns
- **No bootstrapping:** Cannot leverage value estimates for faster learning

Overview: Actor-Critic Methods

Actor-critic methods **combine the best of both worlds**:

Actor (Policy):

- Directly parameterizes policy: $\pi_\theta(a|s)$ (stochastic or deterministic)
- Handles **continuous action spaces** naturally
- Optimized via **policy gradients**

Critic (Value Function):

- Approximates value function: $V_\phi(s)$ or $Q_\phi(s, a)$
- Learned via **TD learning** (bootstrapping)

This enables:

- **Continuous control:** Direct output of continuous actions
- **Sample efficiency:** Learn from individual transitions (like DQN)
- **Reduced variance:** Use critic to reduce policy gradient variance (unlike REINFORCE)
- **Bootstrapping:** Leverage value estimates without waiting for episode completion

This Lecture's Contents:

1. **Stochastic Actor-Critic:**
    - Combining policy gradients with value function critics
2. **Deterministic Policy Gradients (DPG):**
    - How to compute policy gradients for deterministic policies
    - On-Policy Deterministic Actor-Critic
    - Off-Policy Deterministic Actor-Critic (OPDAC)
3. **Deep Deterministic Policy Gradient (DDPG):**
    - Combining DPG with deep neural networks
    - Target networks for both actor and critic
    - Experience replay for continuous control
    - Exploration noise (Ornstein-Uhlenbeck process)
4. **Stabilization Techniques (TD3):**
    - Addressing overestimation bias with Double Q-learning
    - Target policy smoothing to reduce variance
    - Delayed policy updates (TD3)
    - Clipped double Q-learning

Stochastic Actor-Critic Methods

Overview: Combining Value-Based and Policy-Based RL

Core Idea:

Actor-Critic combines the strengths of value-based (Q-learning) and policy-based (Policy Gradient) methods:

Component	Role	Advantage
Actor	Learns policy $\pi_\theta(a\|s)$	Direct policy optimization
Critic	Evaluates $V(s)$ or $Q(s,a)$	Reduces variance in gradient estimates

Key Features:

- **Lower variance** than pure policy gradient (critic provides baseline)
- **Online learning** capability (unlike Monte Carlo methods)
- **Works with discrete and continuous action spaces**
- **More sample efficient** than pure policy gradient methods

(Sutton & Barto, Chapter 13, 2018 (https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf))

Mathematical Foundation: From Policy Gradients to Actor-Critic

Starting Point: Policy Gradient Theorem

The policy gradient theorem (from L3) states that the gradient of expected return can be expressed as:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot G_t \right]$$

where $G_t = \sum_{t'=t}^{T} \gamma^{t'-t} r_{t'}$ is the return (cumulative discounted reward from time t).

Derivation Recap:

Starting from the objective $J(\theta) = \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[\sum_{t=0}^{T} \gamma^t r(s_t, a_t) \right]$, we:

1. Apply log-derivative trick: $\nabla_\theta p_\theta(\tau) = p_\theta(\tau) \nabla_\theta \log p_\theta(\tau)$
2. Decompose trajectory:
   $\log p_\theta(\tau) = \log p(s_0) + \sum_{t=0}^{T} \log \pi_\theta(a_t|s_t) + \sum_{t=0}^{T} \log p(s_{t+1}|s_t, a_t)$
3. Keep only policy-dependent terms: $\nabla_\theta \log p_\theta(\tau) = \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t)$

Actor-Critic: Replacing G_t with $Q^{\pi_\theta}(s, a)$

The REINFORCE algorithm uses Monte Carlo estimates (G_t), which have **high variance**. Actor-critic methods achieve lower variance by replacing the return with the action-value function:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} \left[\nabla_\theta \log \pi_\theta(a|s) \cdot Q^{\pi_\theta}(s, a) \right]$$

Why this works: Since $Q^{\pi_\theta}(s, a) = \mathbb{E}_{\pi_\theta}[G_t | s_t = s, a_t = a]$ by definition, we can replace the high-variance sample G_t with the Q-function without introducing bias.

Actor-Critic Update Rules

Using a learned Q-function $Q_\phi(s, a)$ as the critic, we directly implement the policy gradient:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\left[\nabla_\theta \log \pi_\theta(a|s) \cdot Q_\phi(s, a)\right]$$

Critic Update (Q-Function Approximation):

Learn $Q_\phi(s, a)$ via TD learning with target: $y_t = r_t + \gamma Q_\phi(s_{t+1}, a_{t+1})$

$$\phi \leftarrow \phi - \alpha_\phi \cdot (Q_\phi(s_t, a_t) - y_t) \cdot \nabla_\phi Q_\phi(s_t, a_t)$$

Actor Update (Policy Improvement):

Update policy in direction of Q-value:

$$\theta \leftarrow \theta + \alpha_\theta \cdot Q_\phi(s_t, a_t) \cdot \nabla_\theta \log \pi_\theta(a_t|s_t)$$

Stochastic Actor-Critic Algorithm (Discrete Actions)

Algorithm: Actor-Critic with Discrete Actions
Input: $\alpha_\theta, \alpha_\phi$ (with $\alpha_\phi >> \alpha_\theta$), γ
Output: $\pi_\theta(a|s), Q_\phi(s,a)$
---
1. Initialize actor parameters θ and critic parameters ϕ
2. For episode $= 1, 2, \ldots$:
  3. Observe initial state s_0
  4. For $t = 0, 1, 2, \ldots$:
    5. Sample action: $a_t \sim \pi_\theta(\cdot|s_t)$
    6. Execute a_t, observe r_t, s_{t+1}
    7. Sample next action: $a_{t+1} \sim \pi_\theta(\cdot|s_{t+1})$
    8. Compute TD target: $y_t = r_t + \gamma Q_\phi(s_{t+1}, a_{t+1})$
    9. Update critic: $\phi \leftarrow \phi - \alpha_\phi \cdot (Q_\phi(s_t, a_t) - y_t) \cdot \nabla_\phi Q_\phi(s_t, a_t)$
    10. Update actor: $\theta \leftarrow \theta + \alpha_\theta \cdot Q_\phi(s_t, a_t) \cdot \nabla_\theta \log \pi_\theta(a_t|s_t)$
    11. If terminal, break
12. Return $\pi_\theta(a|s), Q_\phi(s,a)$

Key Steps:

- **Step 5:** Stochastic action selection from policy distribution
- **Step 7-8:** Compute TD target using Q-function (SARSA-style on-policy)
- **Step 9:** Critic learns Q-function via TD learning (fast timescale: $\alpha_\phi >> \alpha_\theta$)
- **Step 10:** Actor improves policy using Q-value as gradient weight (slow timescale)

Two-Timescale Learning:

The critic learns faster than the actor ($\alpha_\phi >> \alpha_\theta$, typically $\alpha_\phi \approx 10 \times \alpha_\theta$):

- **Theoretical motivation:** Ensures Q-function approximates Q^{π_θ} before policy changes, simplifying convergence proofs
- **Practical effect:** Provides more stable Q-value estimates for policy gradients
- **Note:** Modern methods (DDPG, TD3) often use similar learning rates successfully; the strict separation is not always necessary in practice, and there are also convergence proofs that do not require two timescales.

Convergence Properties

Under standard assumptions (diminishing step sizes):

- **Critic** converges to $Q^{\pi_\theta}(s, a)$ via SARSA (on-policy TD learning)
- **Actor** follows the policy gradient and converges to a local optimum of $J(\theta)$

Consider the convergence proof for V-based actor-critic (Sutton & Barto, Section 13.5). It extends naturally to Q-based actor-critic because:

1. SARSA converges to $Q^{\pi_\theta}(s, a)$ under the same conditions as TD(0) converges to $V^{\pi_\theta}(s)$ (both are on-policy TD methods)
2. The policy gradient theorem holds with $Q^{\pi_\theta}(s, a)$ directly (no approximation via TD error needed)
3. The unbiased gradient property is preserved: $\mathbb{E}_{\pi_\theta}[Q^{\pi_\theta}(s, a)\nabla_\theta \log \pi_\theta(a|s)] = \nabla_\theta J(\theta)$

([Sutton & Barto, Sections 6.4 & 13.5, 2018 (https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf)](https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf))

Limitations of Stochastic Actor-Critic

Key Limitations:

1. **Continuous Action Spaces:**
   - Requires **discretization** of continuous actions (e.g., binning torque values)
   - Discretization leads to loss of precision and explosion of action space
   - Example: 10 bins per dimension $\rightarrow 10^d$ actions for d-dimensional action space
2. **Sample Inefficiency:**
   - Stochastic sampling increases variance in gradient estimates
   - Requires many samples to accurately estimate policy gradients
   - Less efficient than deterministic policies in high-dimensional spaces
3. **Exploration-Exploitation Trade-off:**
   - Must maintain sufficient stochasticity for exploration
   - Can be slow to converge in environments requiring precise control
4. **Computational Cost:**
   - Sampling from policy distribution adds overhead
   - Computing log probabilities and entropy regularization increases complexity

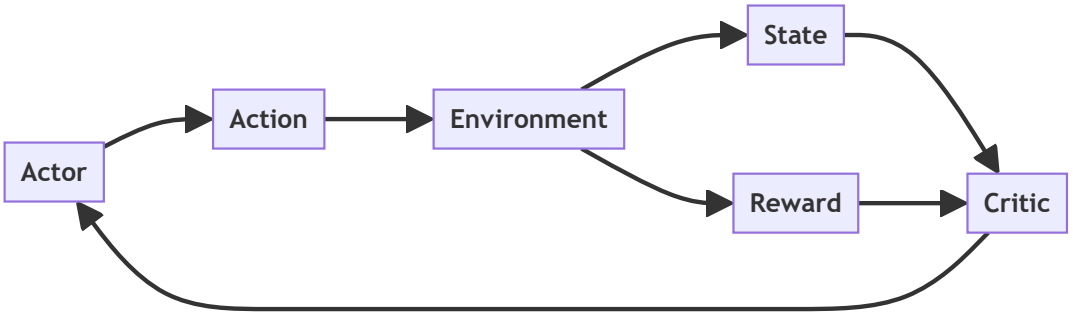
Motivating Deterministic Actor-Critic:

These limitations led to the development of **Deterministic Policy Gradient (DPG)** methods, which:

- Directly output continuous actions without sampling
- More efficient gradient computation in continuous spaces
- Better suited for robotics and continuous control tasks

Deterministic Actor-Critic Methods

Overview and Motivation



Motivation for Deterministic Policies

Deterministic actor-critic methods address the challenge of learning in continuous action spaces where traditional stochastic policy gradients can be inefficient.

(Sutton & Barto, Chapter 13, 2018 (https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf); Silver et al., ICML 2014 (http://proceedings.mlr.press/v32/silver14.pdf))

Key advantages:

- Direct output of continuous actions (no sampling required)
- More efficient gradient estimation in continuous spaces
- Lower computational cost (no integration over action space)

Actor-Critic Architecture

The deterministic actor-critic framework consists of two components:

(Eysenbach, COS435 Lecture 14, 2025 (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-14-ddpg.pdf))

Actor-Critic Structure:

- **Actor:** Deterministic policy $\pi_\theta : \mathcal{S} \to \mathcal{A}$ mapping states directly to actions
- **Critic:** Action-value function $Q_\phi : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ evaluating state-action pairs

Next, we'll Explore:

Deterministic Policy Gradient (DPG): Off-policy extension with separate behavior policy for exploration

Deterministic Policy Gradients (DPG)

Background

Important assumption

DPG assumes a deterministic policy $\pi : \mathcal{S} \mapsto \mathcal{A}$.

Objective

Our goal is to find the optimal policy parameters θ^* that maximize the expected return:

$$J(\theta) = \mathbb{E}_{s \sim \rho^\pi}[Q^\pi(s, \pi_\theta(s))]$$

where $\rho^\pi(s) = \sum_{t=0}^{\infty} \gamma^t P(S_t = s | S_0, \pi)$ is the discounted state visitation distribution.

Policy Gradients for Stochastic Policies (Review)

In L3, we learned how to optimize stochastic policies $\pi_\theta(a|s)$ using the policy gradient theorem. For stochastic policies, we compute gradients based on **rewards** obtained from trajectories:

([Sutton & Barto, Chapter 13, 2018 (https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf)](https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf))

Stochastic Policy Gradient Theorem (REINFORCE):

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a|s) G_t]$$

or with the Q-function:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a|s) Q^\pi(s, a)]$$

Key insight: The $\log \pi_\theta(a|s)$ trick allows us to differentiate through the stochastic policy by weighting returns with log-probabilities.

Policy Gradients for Deterministic Policies

Now we consider **deterministic policies** $\pi_\theta(s)$ that directly output an action (no distribution). This requires a different approach because:

- No probability distribution to sample from: $a = \pi_\theta(s)$ (not $a \sim \pi_\theta(\cdot|s)$)
- The $\log \pi_\theta(a|s)$ trick doesn't apply
- We need a new policy gradient theorem

(Silver et al., ICML 2014 (http://proceedings.mlr.press/v32/silver14.pdf))

Deterministic Policy Gradient Theorem - Derivation

Step 1: Express the objective

Start with the expected Q-value under the deterministic policy:

$$J(\theta) = \mathbb{E}_{s\sim\rho^\pi}[Q^\pi(s, \pi_\theta(s))]$$

Step 2: Take the gradient with respect to policy parameters

$$\nabla_\theta J(\theta) = \nabla_\theta \mathbb{E}_{s\sim\rho^\pi}[Q^\pi(s, \pi_\theta(s))] = \mathbb{E}_{s\sim\rho^\pi}[\nabla_\theta Q^\pi(s, \pi_\theta(s))]$$

Step 3: Apply chain rule to Q-function

The Q-function depends on θ only through the action $a = \pi_\theta(s)$:

$$\nabla_\theta Q^\pi(s, \pi_\theta(s)) = \nabla_a Q^\pi(s, a)|_{a=\pi_\theta(s)} \cdot \nabla_\theta \pi_\theta(s)$$

Step 4: Deterministic Policy Gradient Theorem

Combining steps 2 and 3:

$$\boxed{\nabla_\theta J(\theta) = \mathbb{E}_{s\sim\rho^\pi}[\nabla_a Q^\pi(s, a)|_{a=\pi_\theta(s)} \nabla_\theta \pi_\theta(s)]}$$

Interpretation:

- $\nabla_a Q^\pi(s, a)$: How much Q-value improves by changing the action (critic's guidance)
- $\nabla_\theta \pi_\theta(s)$: How policy action changes with parameters (actor's sensitivity)
- Product: Direction to adjust θ to increase expected return

Key Difference from Stochastic Policy Gradients

The deterministic policy gradient relies on the **Q-function gradient** $\nabla_a Q(s, a)$, **not on rewards** G_t or r_t:

Aspect	Stochastic (REINFORCE)	Deterministic (DPG)
Policy	$\pi_\theta(a\|s)$ (distribution)	$\pi_\theta(s)$ (deterministic)
Gradient based on	**Rewards** G_t weighted by $\nabla_\theta \log \pi_\theta(a\|s)$	**Q-function gradient** $\nabla_a Q^\pi(s, a)$
Sampling	Sample actions from $\pi_\theta(\cdot\|s)$	Direct action $a = \pi_\theta(s)$
Variance	High (Monte Carlo returns)	Lower (bootstrapped Q-values)
Critic required?	Optional (reduces variance)	**Essential** (provides $\nabla_a Q$)

Why this matters:

- **REINFORCE** learns from **actual returns** G_t obtained by following the policy until episode end
- **DPG** learns from **Q-function estimates** $Q^\pi(s, a)$ which can be updated online via TD learning
- **Result:** DPG is more sample-efficient and works in continuing (non-episodic) tasks

([Eysenbach, COS435 Lecture 14, 2025 (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-14-ddpg.pdf)](https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-14-ddpg.pdf))

Deterministic Online Actor-Critic (On-Policy)

The simplest deterministic actor-critic approach follows the **on-policy** paradigm, similar to SARSA from L6. The policy directly interacts with the environment and learns from its own experience.

(Sutton & Barto, Chapter 13, 2018 (https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf))

Key Characteristics:

- **Policy = Behavior:** The same deterministic policy $\pi_\theta(s)$ is used for both action selection and learning
- **Online Updates:** Critic and actor update after each transition
- **SARSA-style:** Uses actual next action from policy for TD target

Update Equations:

Critic Update (SARSA TD target):

$$Q_\phi(s_t, a_t) \leftarrow Q_\phi(s_t, a_t) + \alpha_\phi \delta_t$$

where the TD error is:

$$\delta_t = r_t + \gamma Q_\phi(s_{t+1}, \pi_\theta(s_{t+1})) - Q_\phi(s_t, a_t)$$

Actor Update (Policy Gradient):

$$\theta \leftarrow \theta + \alpha_\theta \nabla_a Q_\phi(s_t, a)|_{a=\pi_\theta(s_t)} \nabla_\theta \pi_\theta(s_t)$$

Algorithm: Deterministic Online Actor-Critic (SARSA-style)

Algorithm: Deterministic Online Actor-Critic
Input: Learning rates $\alpha_\theta, \alpha_\phi;$ discount factor γ
Output: Learned deterministic policy $\pi_\theta(s)$ and Q-function $Q_\phi(s, a)$
---
1. Initialize actor π_θ and critic Q_ϕ with random parameters
2. For episode $= 1$ to M :
  3. Observe initial state s_0
  4. For $t = 0, 1, 2, \ldots$:
    5. Select action: $a_t = \pi_\theta(s_t)$
    6. Execute $a_t,$ observe r_t, s_{t+1}
    7. Compute TD error: $\delta_t = r_t + \gamma Q_\phi(s_{t+1}, \pi_\theta(s_{t+1})) - Q_\phi(s_t, a_t)$
    8. Update critic: $\phi \leftarrow \phi + \alpha_\phi \delta_t \nabla_\phi Q_\phi(s_t, a_t)$
    9. Update actor: $\theta \leftarrow \theta + \alpha_\theta \nabla_a Q_\phi(s_t, a)|_{a=\pi_\theta(s_t)} \nabla_\theta \pi_\theta(s_t)$
    10. $s_t \leftarrow s_{t+1}$
    11. If terminal, break
12. Return $\pi_\theta(s), Q_\phi(s, a)$

Limitations of On-Policy Approach:

1. **No Exploration:** Deterministic policy always selects same action in same state
2. **Sample Inefficiency:** Can only learn from most recent experience
3. **Unstable Learning:** Highly correlated sequential data

These limitations motivate the **off-policy extension** we'll see next.

Off-Policy Deterministic Actor-Critic (OPDAC)

To address the exploration problem, we introduce a **noisy behavior policy** $b(a|s)$ that differs from the deterministic target policy $\pi_\theta(s)$. This enables **off-policy learning**.

(Silver et al., ICML 2014 (http://proceedings.mlr.press/v32/silver14.pdf))

Key Innovation: Noisy Behavior Policy

We define a stochastic behavior policy for exploration:

$$b(a|s) = \pi_\theta(s) + \mathcal{N}(0, \sigma^2)$$

where:

- **Target policy** $\pi_\theta(s)$: Deterministic policy we want to learn (exploitation)
- **Behavior policy** $b(a|s)$: Stochastic policy used for data collection (exploration)
- **Noise** $\mathcal{N}(0, \sigma^2)$: Gaussian noise for exploration

Algorithm: Off-Policy Deterministic Actor-Critic (OPDAC)

(Silver et al., ICML 2014 (http://proceedings.mlr.press/v32/silver14.pdf))

Algorithm: Off-Policy Deterministic Actor-Critic

Input: $\alpha_\theta, \alpha_\phi, \gamma, \sigma^2$

Output: $\pi_\theta(s), Q_\phi(s,a)$

---

1. Initialize actor $\pi_\theta,$ critic Q_ϕ

2. For episode $= 1, 2, \ldots :$

  3. Observe initial state s_0

  4. For $t = 0, 1, 2, \ldots :$

    5. Select action with noise (behavior policy): $a_t = \pi_\theta(s_t) + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, \sigma^2)$

    6. Execute $a_t,$ observe r_t, s_{t+1}

    7. Compute TD target: $y_t = r_t + \gamma Q_\phi(s_{t+1}, \pi_\theta(s_{t+1}))$

    8. Update critic: $\phi \leftarrow \phi - \alpha_\phi \nabla_\phi (Q_\phi(s_t, a_t) - y_t)^2$

    9. Update actor: $\theta \leftarrow \theta + \alpha_\theta \nabla_a Q_\phi(s_t, a)|_{a=\pi_\theta(s_t)} \nabla_\theta \pi_\theta(s_t)$

    10. If terminal, break

11. Return $\pi_\theta(s), Q_\phi(s,a)$

Key Features:

- **Online updates:** Critic and actor updated immediately after each transition
- **Off-policy learning:** Uses behavior policy $b(a|s) = \pi_\theta(s) + \mathcal{N}(0, \sigma^2)$ for exploration|

Test Environment: Pendulum-v1

We test the OPDAC algorithm on the classic **Pendulum-v1** environment from Gymnasium.

Environment Description:

- **Goal:** Swing up an inverted pendulum and balance it upright
- **State Space:** 3-dimensional continuous
  - $\cos(\theta)$: cosine of angle from vertical
  - $\sin(\theta)$: sine of angle from vertical
  - $\dot{\theta}$: angular velocity
- **Action Space:** 1-dimensional continuous, $a \in [-2, 2]$
  - Torque applied to the pendulum
- **Reward Function:**

$$r_t = -(\theta^2 + 0.1 \cdot \dot{\theta}^2 + 0.001 \cdot a^2)$$

  - Penalizes deviation from upright position ($\theta = 0$)
  - Penalizes high angular velocity
  - Penalizes large control effort
  - Maximum reward per step: 0 (when perfectly balanced with no torque)
  - Typical range: $[-16.27, 0]$
- **Episode Length:** 200 time steps

Source: [Gymnasium Documentation (https://gymnasium.farama.org/environments/classic_control/pendulum/)](https://gymnasium.farama.org/environments/classic_control/pendulum/)

In [1]:

```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import importlib
import actor_critic_helpers
importlib.reload(actor_critic_helpers)
from actor_critic_helpers import (plot_training_progress, render_
episode,
                                  Actor, Critic, select_action_w
ith_noise)


def compute_td_target(critic, actor, reward, next_state, done, ga
mma=0.99):
    """
    Compute TD target: y = r + γ Q_φ(s', π_ϑ(s'))

    Args:
        critic: Critic network
        actor: Actor network
        reward: Reward received
        next_state: Next state
        done: Whether episode is done
        gamma: Discount factor

    Returns:
        TD target as scalar
    """
    with torch.no_grad():
        next_state_tensor = torch.FloatTensor(next_state).unsquee
ze(0)
        next_action = actor(next_state_tensor)
        next_q_value = critic(next_state_tensor, next_action).ite
m()

        # y = r + γ Q(s', π(s')) for non-terminal states
        target = reward + gamma * next_q_value * (1 - done)

    return target


def update_critic(critic, critic_optimizer, state, action, targe
t):
    """
    Update critic using MSE loss: φ ← φ - α_φ ∇_φ (Q_φ(s,a) - y)²

    Args:
        critic: Critic network
        critic_optimizer: Optimizer for critic
        state: Current state
        action: Action taken
        target: TD target

    Returns:
        Critic loss value
```

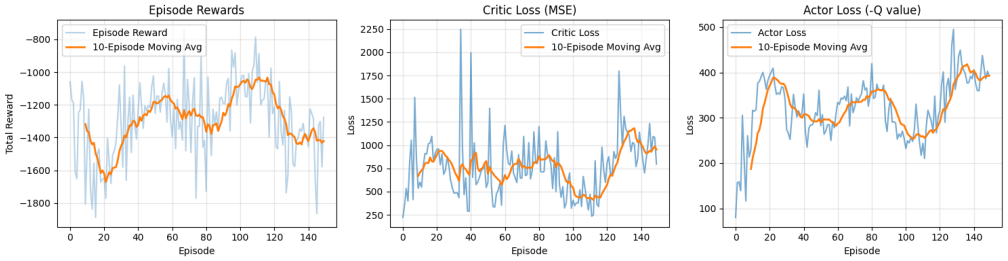
```
    """
    state_tensor = torch.FloatTensor(state).unsqueeze(0)
    action_tensor = torch.FloatTensor(action).unsqueeze(0)
    target_tensor = torch.FloatTensor([target])

    # Compute Q-value
    q_value = critic(state_tensor, action_tensor)

    # Compute MSE loss
    critic_loss = nn.MSELoss()(q_value, target_tensor)

    # Gradient descent
    critic_optimizer.zero_grad()
    critic_loss.backward()
    critic_optimizer.step()

    return critic_loss.item()


def update_actor(actor, actor_optimizer, critic, state):
    """
    Update actor using DPG: ϑ ← ϑ + α_ϑ ∇_a Q_φ|_{a=π_ϑ(s)} ∇_ϑ π
_ϑ(s)

    Args:
        actor: Actor network
        actor_optimizer: Optimizer for actor
        critic: Critic network
        state: Current state

    Returns:
        Actor loss value
    """
    state_tensor = torch.FloatTensor(state).unsqueeze(0)

    # Compute action from current policy
    action = actor(state_tensor)

    # Actor loss is negative Q-value (gradient ascent on Q)
    actor_loss = -critic(state_tensor, action)

    # Gradient ascent (negative of gradient descent)
    actor_optimizer.zero_grad()
    actor_loss.backward()
    actor_optimizer.step()

    return actor_loss.item()


def opdac(env, state_dim, action_dim,
          num_episodes=1000, max_steps=1000,
          gamma=0.99, actor_lr=1e-4, critic_lr=1e-3,
          noise_std=0.1, plot_freq=10, render_env=None, render_fr
eq=None):
    """
    Off-Policy Deterministic Actor-Critic (OPDAC) Algorithm
```

```
    Implements the algorithm from the pseudocode:
    1. Initialize actor π_ϑ, critic Q_φ
    2. For each episode:
    3.   Reset environment, get initial state s_0
    4.   For each time step:
    5.     Select action with noise: a_t = π_ϑ(s_t) + ε
    6.     Execute action, observe reward r_t and next state s_{t
+1}
    7.     Compute TD target: y = r_t + γ Q_φ(s_{t+1}, π_ϑ(s_{t+
1}))
    8.     Update critic: φ ← φ - α_φ ∇_φ (Q_φ(s_t,a_t) - y)²
    9.     Update actor: ϑ ← ϑ + α_ϑ ∇_a Q_φ|_{a=π_ϑ(s_t)} ∇_ϑ π_
ϑ(s_t)
    10.    If terminal, break
    11. Return actor and critic networks

    Args:
        env: Gym environment
        state_dim: Dimension of state space
        action_dim: Dimension of action space
        num_episodes: Number of training episodes
        max_steps: Maximum steps per episode
        gamma: Discount factor
        actor_lr: Learning rate for actor
        critic_lr: Learning rate for critic
        noise_std: Standard deviation of exploration noise
        plot_freq: Frequency of plotting updates
        render_env: Optional environment for rendering during tra
ining
        render_freq: Frequency of rendering episodes (if render_e
nv provided)

    Returns:
        Tuple of (actor, critic, episode_rewards, actor_losses, c
ritic_losses)
    """
    # Step 1: Initialize networks
    actor = Actor(state_dim, action_dim)
    critic = Critic(state_dim, action_dim)
    actor_optimizer = optim.Adam(actor.parameters(), lr=actor_lr)
    critic_optimizer = optim.Adam(critic.parameters(), lr=critic_
lr)

    episode_rewards = []
    episode_actor_losses = []
    episode_critic_losses = []

    # Step 2: For each episode
    for episode in range(num_episodes):
        # Step 3: Reset environment (gymnasium returns state, inf
o)
        state, info = env.reset()
        episode_reward = 0
        actor_losses = []
        critic_losses = []

        # Step 4: For each time step
```

```
        for t in range(max_steps):
            # Step 5: Select action with exploration noise (behav
ior policy b)
            action = select_action_with_noise(actor, state, noise
_std)

            # Step 6: Execute action and observe (gymnasium retur
ns 5 values)
            next_state, reward, terminated, truncated, info = en
v.step(action)
            done = terminated or truncated
            episode_reward += reward

            # Step 7: Compute TD target
            target = compute_td_target(critic, actor, reward, nex
t_state, done, gamma)

            # Step 8: Update critic
            critic_loss = update_critic(critic, critic_optimizer,
state, action, target)
            critic_losses.append(critic_loss)

            # Step 9: Update actor using deterministic policy gra
dient
            actor_loss = update_actor(actor, actor_optimizer, cri
tic, state)
            actor_losses.append(actor_loss)

            state = next_state

            # Step 10: Check if terminal
            if done:
                break

        # Store episode statistics
        episode_rewards.append(episode_reward)
        episode_actor_losses.append(np.mean(actor_losses))
        episode_critic_losses.append(np.mean(critic_losses))

        # Plot progress
        if (episode + 1) % plot_freq == 0:
            plot_training_progress(episode_rewards, episode_actor
_losses,
                                   episode_critic_losses)

        # Render episode if requested
        if render_env is not None and render_freq is not None:
            if (episode + 1) % render_freq == 0:
                print(f"\nRendering episode {episode + 1}...")
                render_reward = render_episode(render_env, actor,
max_steps)
                print(f"Render episode reward: {render_reward:.2
f}")

    # Step 11: Return trained networks and statistics
    return actor, critic, episode_rewards, episode_actor_losses,
episode_critic_losses
```

In [2]:

```
import gymnasium as gym
import numpy as np
from actor_critic_helpers import plot_rendered_episode

# Create two environments: one for training, one for rendering
env_train = gym.make('Pendulum-v1')
env_render = gym.make('Pendulum-v1', render_mode='human')

# Get environment dimensions
state_dim = env_train.observation_space.shape[0]
action_dim = env_train.action_space.shape[0]

print(f"Environment: Pendulum-v1")
print(f"State dimension: {state_dim}")
print(f"Action dimension: {action_dim}")
print(f"Action space: [{env_train.action_space.low[0]:.1f}, {env_
train.action_space.high[0]:.1f}]")
print()

# Train OPDAC with improved hyperparameters
print("Starting OPDAC training...")
actor, critic, rewards, actor_losses, critic_losses = opdac(
    env_train,
    state_dim,
    action_dim,
    num_episodes=150,          # More episodes for better converg
ence
    max_steps=200,
    gamma=0.99,
    actor_lr=3e-4,             # Increased from 1e-4 (faster acto
r updates)
    critic_lr=1e-3,            # Keep critic learning rate
    noise_std=0.3,             # Increased from 0.2 (more explora
tion initially)
    plot_freq=10,
    render_env=env_render,
    render_freq=50             # Render every 50 episodes
)

print("\nTraining complete!")
print(f"Final 10-episode average reward: {np.mean(rewards[-1
0:]):.2f}")
print(f"Best episode reward: {np.max(rewards):.2f}")

# Render final episodes with the trained policy
print("\nRendering final trained policy...")
for i in range(3):
    print(f"\nFinal Test Episode {i+1}:")
    reward = plot_rendered_episode(env_render, actor, state_dim,
action_dim, max_steps=200)
    print(f"Total Reward: {reward:.2f}")

env_train.close()
env_render.close()
```




```
Rendering episode 150...
Render episode reward: -1380.64

Training complete!
Final 10-episode average reward: -1421.47
Best episode reward: -738.60

Rendering final trained policy...

Final Test Episode 1:
Total Reward: -1349.60

Final Test Episode 2:
Total Reward: -1306.27

Final Test Episode 3:
Total Reward: -1248.31
```

Benefits and Limitations of Deterministic Actor-Critic

Benefits over Pure Policy Gradient Methods:

- **Lower variance:** Critic provides a baseline, reducing gradient variance compared to REINFORCE
- **Sample efficiency:** Can learn from individual transitions, not just complete episodes
- **Continuous actions:** Naturally handles continuous action spaces without discretization
- **Computational efficiency:** Deterministic policy gradient is simpler than stochastic policy gradient (no integration over actions)

Benefits over Q-Learning:

- **Continuous actions:** No need to discretize action space or maximize over actions
- **Direct policy representation:** Policy $\pi_\theta(s)$ is explicitly parameterized
- **Smooth policies:** Network outputs smooth action mappings

Limitations of OPDAC:

- **Training instability:** Online updates can lead to unstable learning dynamics
  - Critic's Q-values change rapidly as it learns
  - Actor updates using a moving target (changing critic)
  - Strong correlations between consecutive samples
- **Sensitivity to hyperparameters:** Learning rates, exploration noise require careful tuning
- **Slow convergence:** Online learning from single transitions can be inefficient
- **Overestimation bias:** Critic may overestimate Q-values, leading to poor policy updates
- **Brittle exploration:** Simple additive noise may be insufficient for complex tasks

These challenges motivate the development of more robust algorithms that build upon the deterministic actor-critic framework...

Deep Deterministic Policy Gradient (DDPG)

Overview of Deep Deterministic Policy Gradient

DDPG (Lillicrap et al., ICLR 2016 (https://arxiv.org/abs/1509.02971)) extends the deterministic policy gradient algorithm with two core algorithmic improvements and additional engineering techniques.

Core Algorithmic Features:

1. **Experience Replay:** Store transitions in a replay buffer and sample mini-batches for training
    - Breaks temporal correlations between consecutive samples
    - Enables better sample efficiency through repeated use of experiences
2. **Target Networks:** Maintain slowly-updating copies of actor and critic networks
    - Provides stable target values during training
    - Reduces oscillations and divergence in learning

Additional Engineering Features:

1. **Exploration Noise:** Add Gaussian noise to actions
2. **Batch Normalization:** Normalize inputs to network layers for stable training across diverse environments

These modifications address the instability and sample inefficiency issues of basic deterministic actor-critic methods.

Algorithm: Deep Deterministic Policy Gradient (DDPG)

Input: $\alpha_\theta, \alpha_\phi, \gamma, \sigma^2, \tau, N, B$

Output: $\pi_\theta(s), Q_\phi(s,a)$

---

1. Initialize actor $\pi_\theta,$ critic Q_ϕ

2. Initialize target networks: $\pi_{\theta'} \leftarrow \pi_\theta,\ Q_{\phi'} \leftarrow Q_\phi$

3. Initialize replay buffer $\mathcal{D}$ with capacity N

4. For episode $= 1, 2, \ldots :$

  5. Observe initial state s_0

  6. For $t = 0, 1, 2, \ldots :$

    7. Select action with noise: $a_t = \pi_\theta(s_t) + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, \sigma^2)$

    8. Execute $a_t,$ observe r_t, s_{t+1}

    9. Store transition (s_t, a_t, r_t, s_{t+1}) in $\mathcal{D}$

    10. Sample random minibatch of B transitions from $\mathcal{D}$

    11. For each transition in minibatch:

      Compute TD target using target networks: $y_i = r_i + \gamma Q_{\phi'}(s_{i+1}, \pi_{\theta'}(s_{i+1}))$

    12. Update critic: $\phi \leftarrow \phi - \alpha_\phi \dfrac{1}{B} \sum_i \nabla_\phi (Q_\phi(s_i, a_i) - y_i)^2$

    13. Update actor: $\theta \leftarrow \theta + \alpha_\theta \dfrac{1}{B} \sum_i \nabla_a Q_\phi(s_i, a)|_{a=\pi_\theta(s_i)} \nabla_\theta \pi_\theta(s_i)$

    14. Soft update target networks:

      $\theta' \leftarrow \tau\theta + (1-\tau)\theta'$

      $\phi' \leftarrow \tau\phi + (1-\tau)\phi'$

    15. If terminal, break

16. Return $\pi_\theta(s), Q_\phi(s,a)$

Key Changes from OPDAC (highlighted in blue):

- **Target networks** $\pi_{\theta'}$ and $Q_{\phi'}$ with soft updates using $\tau \ll 1$ (typically 0.001)
- **Experience replay buffer** $\mathcal{D}$ stores transitions for off-policy learning
- **Minibatch updates** of size B (typically 64-256) instead of single-transition updates
- **Decoupled data collection and learning:** Transitions stored first, then sampled for training

Benefits:

- Improved stability through slowly-updating target networks
- Better sample efficiency through experience replay
- Reduced correlation in training data via random sampling

In [3]:

```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import importlib
import actor_critic_helpers
importlib.reload(actor_critic_helpers)
from actor_critic_helpers import (plot_training_progress, render_
episode,
                                  Actor, Critic, select_action_w
ith_noise, ReplayBuffer)


def compute_td_target_batch(critic_target, actor_target, rewards,
next_states, dones, gamma=0.99):
    """
    Compute TD targets for a batch using target networks: y = r +
γ Q_φ'(s', π_ϑ'(s'))

    Args:
        critic_target: Target critic network
        actor_target: Target actor network
        rewards: Batch of rewards
        next_states: Batch of next states
        dones: Batch of done flags
        gamma: Discount factor

    Returns:
        TD targets as tensor
    """
    with torch.no_grad():
        next_states_tensor = torch.FloatTensor(next_states)
        next_actions = actor_target(next_states_tensor)
        next_q_values = critic_target(next_states_tensor, next_ac
tions).squeeze()

        # y = r + γ Q'(s', π'(s')) for non-terminal states
        targets = torch.FloatTensor(rewards) + gamma * next_q_val
ues * (1 - torch.FloatTensor(dones))

    return targets


def update_critic_batch(critic, critic_optimizer, states, action
s, targets):
    """
    Update critic using batch MSE loss: φ ← φ - α_φ (1/B) Σ ∇_φ
(Q_φ(s,a) - y)²

    Args:
        critic: Critic network
        critic_optimizer: Optimizer for critic
        states: Batch of states
        actions: Batch of actions
        targets: Batch of TD targets
```

```
    Returns:
        Critic loss value
    """
    states_tensor = torch.FloatTensor(states)
    actions_tensor = torch.FloatTensor(actions)

    # Compute Q-values
    q_values = critic(states_tensor, actions_tensor).squeeze()

    # Compute MSE loss
    critic_loss = nn.MSELoss()(q_values, targets)

    # Gradient descent
    critic_optimizer.zero_grad()
    critic_loss.backward()
    critic_optimizer.step()

    return critic_loss.item()


def update_actor_batch(actor, actor_optimizer, critic, states):
    """
    Update actor using batch DPG: ϑ ← ϑ + α_ϑ (1/B) Σ ∇_a Q_φ|_{a
=π_ϑ(s)} ∇_ϑ π_ϑ(s)

    Args:
        actor: Actor network
        actor_optimizer: Optimizer for actor
        critic: Critic network
        states: Batch of states

    Returns:
        Actor loss value
    """
    states_tensor = torch.FloatTensor(states)

    # Compute actions from current policy
    actions = actor(states_tensor)

    # Actor loss is negative mean Q-value (gradient ascent on Q)
    actor_loss = -critic(states_tensor, actions).mean()

    # Gradient ascent
    actor_optimizer.zero_grad()
    actor_loss.backward()
    actor_optimizer.step()

    return actor_loss.item()


def soft_update_target_networks(source, target, tau):
    """
    Soft update target networks: ϑ' ← τϑ + (1-τ)ϑ'

    Args:
        source: Source network (actor or critic)
        target: Target network
```

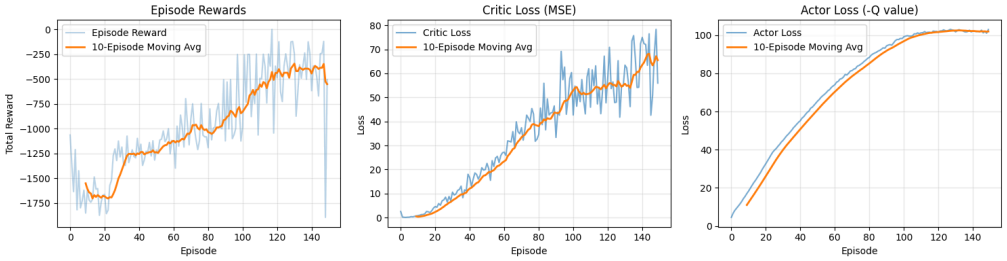
```
        tau: Soft update parameter (typically 0.001)
    """
    for target_param, source_param in zip(target.parameters(), so
urce.parameters()):
        target_param.data.copy_(tau * source_param.data + (1 - ta
u) * target_param.data)


def ddpg(env, state_dim, action_dim,
         num_episodes=1000, max_steps=1000,
         gamma=0.99, actor_lr=1e-4, critic_lr=1e-3,
         noise_std=0.1, tau=0.001, buffer_capacity=100000, batch_
size=64,
         plot_freq=10, render_env=None, render_freq=None):
    """
    Deep Deterministic Policy Gradient (DDPG) Algorithm

    Implements the algorithm from the pseudocode:
    1. Initialize actor π_ϑ, critic Q_φ
    2. Initialize target networks: π_ϑ' ← π_ϑ, Q_φ' ← Q_φ
    3. Initialize replay buffer D with capacity N
    4. For each episode:
    5.   Reset environment, get initial state s_0
    6.   For each time step:
    7.     Select action with noise: a_t = π_ϑ(s_t) + ε
    8.     Execute action, observe reward r_t and next state s_{t
+1}
    9.     Store transition (s_t, a_t, r_t, s_{t+1}) in D
    10.    Sample random minibatch of B transitions from D
    11.    For each transition in minibatch:
           Compute TD target using target networks: y_i = r_i + γ
Q_φ'(s_{i+1}, π_ϑ'(s_{i+1}))
    12.    Update critic: φ ← φ - α_φ (1/B) Σ ∇_φ (Q_φ(s_i,a_i) -
y_i)²
    13.    Update actor: ϑ ← ϑ + α_ϑ (1/B) Σ ∇_a Q_φ|_{a=π_ϑ(s_
i)} ∇_ϑ π_ϑ(s_i)
    14.    Soft update target networks: ϑ' ← τϑ + (1-τ)ϑ', φ' ← τ
φ + (1-τ)φ'
    15.    If terminal, break
    16. Return actor and critic networks

    Args:
        env: Gym environment
        state_dim: Dimension of state space
        action_dim: Dimension of action space
        num_episodes: Number of training episodes
        max_steps: Maximum steps per episode
        gamma: Discount factor
        actor_lr: Learning rate for actor
        critic_lr: Learning rate for critic
        noise_std: Standard deviation of exploration noise
        tau: Soft update parameter for target networks
        buffer_capacity: Maximum capacity of replay buffer
        batch_size: Minibatch size for training
        plot_freq: Frequency of plotting updates
        render_env: Optional environment for rendering during tra
ining
```

```
        render_freq: Frequency of rendering episodes (if render_e
nv provided)

    Returns:
        Tuple of (actor, critic, episode_rewards, actor_losses, c
ritic_losses)
    """
    # Step 1: Initialize networks
    actor = Actor(state_dim, action_dim)
    critic = Critic(state_dim, action_dim)
    actor_optimizer = optim.Adam(actor.parameters(), lr=actor_lr)
    critic_optimizer = optim.Adam(critic.parameters(), lr=critic_
lr)

    # Step 2: Initialize target networks
    actor_target = Actor(state_dim, action_dim)
    critic_target = Critic(state_dim, action_dim)
    actor_target.load_state_dict(actor.state_dict())
    critic_target.load_state_dict(critic.state_dict())

    # Step 3: Initialize replay buffer
    replay_buffer = ReplayBuffer(buffer_capacity)

    episode_rewards = []
    episode_actor_losses = []
    episode_critic_losses = []

    # Step 4: For each episode
    for episode in range(num_episodes):
        # Step 5: Reset environment
        state, info = env.reset()
        episode_reward = 0
        actor_losses = []
        critic_losses = []

        # Step 6: For each time step
        for t in range(max_steps):
            # Step 7: Select action with exploration noise
            action = select_action_with_noise(actor, state, noise
_std)

            # Step 8: Execute action and observe
            next_state, reward, terminated, truncated, info = en
v.step(action)
            done = terminated or truncated
            episode_reward += reward

            # Step 9: Store transition in replay buffer
            replay_buffer.push(state, action, reward, next_state,
done)

            # Step 10-14: Update networks if enough samples in bu
ffer
            if len(replay_buffer) >= batch_size:
                # Step 10: Sample random minibatch
                states, actions, rewards, next_states, dones = re
play_buffer.sample(batch_size)
```

```
                # Step 11: Compute TD targets using target networ
ks
                targets = compute_td_target_batch(critic_target,
actor_target,
                                                  rewards, next_st
ates, dones, gamma)

                # Step 12: Update critic
                critic_loss = update_critic_batch(critic, critic_
optimizer,
                                                  states, actions,
targets)
                critic_losses.append(critic_loss)

                # Step 13: Update actor
                actor_loss = update_actor_batch(actor, actor_opti
mizer, critic, states)
                actor_losses.append(actor_loss)

                # Step 14: Soft update target networks
                soft_update_target_networks(actor, actor_target,
tau)
                soft_update_target_networks(critic, critic_targe
t, tau)

            state = next_state

            # Step 15: Check if terminal
            if done:
                break

        # Store episode statistics
        episode_rewards.append(episode_reward)
        if len(actor_losses) > 0:
            episode_actor_losses.append(np.mean(actor_losses))
            episode_critic_losses.append(np.mean(critic_losses))
        else:
            episode_actor_losses.append(0)
            episode_critic_losses.append(0)

        # Plot progress
        if (episode + 1) % plot_freq == 0:
            plot_training_progress(episode_rewards, episode_actor
_losses,
                                   episode_critic_losses)

        # Render episode if requested
        if render_env is not None and render_freq is not None:
            if (episode + 1) % render_freq == 0:
                print(f"\nRendering episode {episode + 1}...")
                render_reward = render_episode(render_env, actor,
max_steps)
                print(f"Render episode reward: {render_reward:.2
f}")

    # Step 16: Return trained networks and statistics
```

```
    return actor, critic, episode_rewards, episode_actor_losses,
episode_critic_losses
```

In [4]:
```python
import gymnasium as gym
import numpy as np
from actor_critic_helpers import plot_rendered_episode

# Create two environments: one for training, one for rendering
env_train = gym.make('Pendulum-v1')
env_render = gym.make('Pendulum-v1', render_mode='human')

# Get environment dimensions
state_dim = env_train.observation_space.shape[0]
action_dim = env_train.action_space.shape[0]

print(f"State dimension: {state_dim}")
print(f"Action dimension: {action_dim}")

# Train DDPG
actor, critic, rewards_ddpg, actor_losses, critic_losses = ddpg(
    env=env_train,
    state_dim=state_dim,
    action_dim=action_dim,
    num_episodes=150,
    max_steps=200,
    gamma=0.99,
    actor_lr=3e-4,
    critic_lr=1e-3,
    noise_std=0.3,
    tau=0.001,
    buffer_capacity=100000,
    batch_size=64,
    plot_freq=10,
    render_env=env_render,
    render_freq=50
)

print("\nDDPG Training completed!")
print(f"Final average reward (last 10 episodes): {np.mean(rewards
_ddpg[-10:]):.2f}")

# Close environments
env_train.close()
env_render.close()
```




```
Rendering episode 150...
Render episode reward: -508.38

DDPG Training completed!
Final average reward (last 10 episodes): -551.66
```

Stability and convergence in DDPG

Despite the improvements from target networks and experience replay, DDPG still faces stability challenges and can be sensitive to hyperparameters.

(CS285 Lecture 23: Challenges and Open Problems in Deep Reinforcement Learning, Sergey Levine, 2023, p. 1-26 (https://rail.eecs.berkeley.edu/deeprlcourse/deeprlcourse/static/slides/lec-23.pdf), search for "stability: does your algorithm converge")

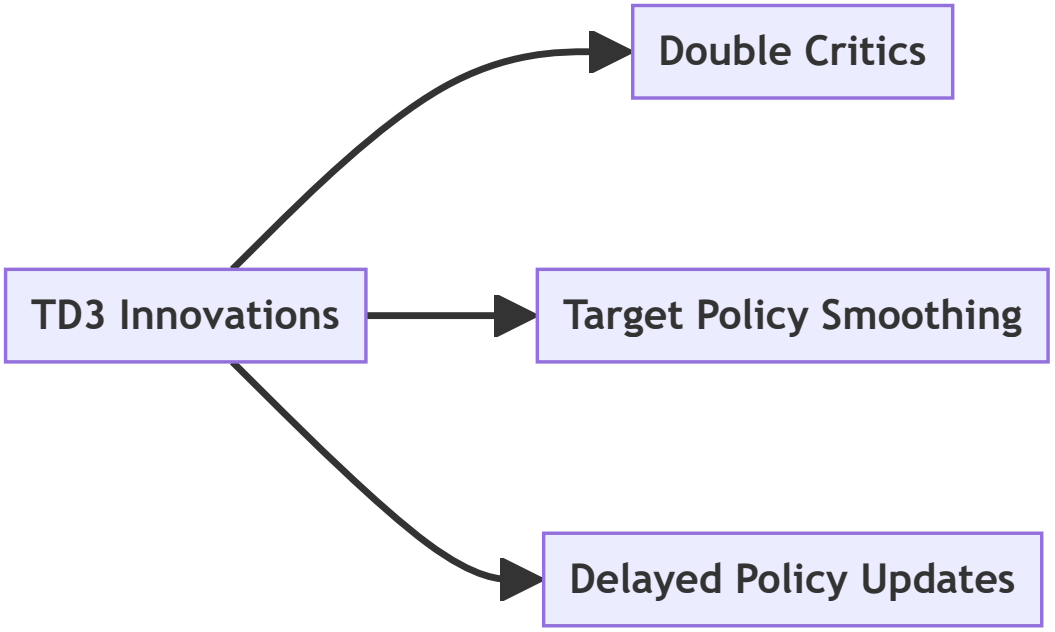
(COS435/ ECE 433: Introduction to RL Lecture 14: Actor Critic Methods, Ben Eysenbach, 2025, p. 2-6 (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-14-ddpg.pdf), search for "target network delay, replay buffer size")

Remaining gaps in DDPG:

- **Q-value overestimation:** The critic can overestimate action values, leading to suboptimal policy updates (cf. cliff example for Q-Learning lecture)
- **Brittle convergence:** Small changes in hyperparameters can cause training to fail
- **High variance in policy updates:** Single critic estimates can be noisy and unstable

Twin Delayed DDPG (TD3)

Overview of Twin Delayed DDPG



TD3 addresses the remaining limitations in DDPG through **three** key innovations (that's is why it is called TD**3**):

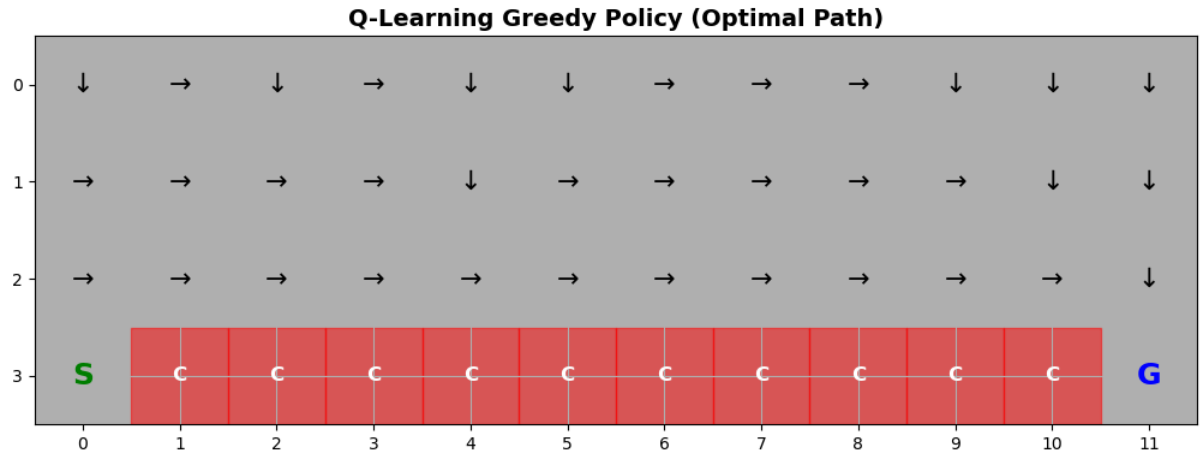
([COS435/ ECE 433: Introduction to RL - Lecture 15: Advanced Actor Critic Methods, Ben Eysenbach, 2025, p. 1-6 (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-15-td3.pdf)](https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-15-td3.pdf), search for "TD3 sought to address was critic overestimation")

DDPG Problem	TD3 Innovation	Description
Q-value overestimation	Double Critics	Use minimum of two Q-networks to reduce overestimation bias
High variance in policy updates	Target Policy Smoothing	Add clipped noise to target actions for regularization
Brittle convergence	Delayed Policy Updates	Update actor less frequently than critic for stability

TD3 Innovation 1: Double Critics

The Overestimation Bias Problem

Standard Q-learning and actor-critic methods suffer from overestimation bias: The max operator in the Bellman backup consistently overestimates Q-values. As an example, re-consider the Cliff - example from the Q-Learning lecture.



Overstimation leads to:

- **Accumulated errors:** Overestimations propagate through bootstrapping
- **Unstable learning:** Policy chases inflated value estimates
- **Poor performance:** Agent overconfident in suboptimal actions

Recall: The cliff problem, where we discussed SARSA vs. Q-Learning On-policy agent choses safe path above the cliff, off-policy agent takes riskier path directly at the cliff.

The Double Q-Learning Solution

TD3 maintains **two independent Q-networks** (critics) $Q_{\phi_1}(s, a)$ and $Q_{\phi_2}(s, a)$ with separate parameters ϕ_1 and ϕ_2.

Target Calculation with Minimum Trick

Instead of using a single overestimated Q-value, TD3 computes the target using the **minimum** of both critics:

$$y = r + \gamma \min_{i=1,2} Q_{\phi_i'}(s', \pi_{\theta'}(s'))$$

where:

- r is the observed reward
- γ is the discount factor
- $\pi_{\theta'}(s')$ is the target actor's action for next state s'
- $Q_{\phi_i'}$ are the target critic networks (soft-updated from Q_{ϕ_i})
- $\min_{i=1,2}$ selects the lower (more conservative) Q-estimate

Why the Minimum Works

The minimum operator provides an **underestimation bias** that counteracts the natural overestimation:

- If Q_{ϕ_1} overestimates, Q_{ϕ_2} likely gives more realistic value
- Taking minimum prevents propagation of the highest noise
- Both critics must agree on high values for them to persist

Critic Loss Functions

Each critic is trained independently using mean squared Bellman error:

$$L(\phi_1) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(Q_{\phi_1}(s,a) - y\right)^2\right]$$

$$L(\phi_2) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(Q_{\phi_2}(s,a) - y\right)^2\right]$$

where $\mathcal{D}$ is the replay buffer and y is the shared target (using the minimum).

Key Insight: Both critics learn from the **same conservative target** (computed with the min), but maintain **independent function approximators**, preventing correlated overestimation errors.

(COS435/ ECE 433: Introduction to RL - Lecture 15: Advanced Actor Critic Methods, Ben Eysenbach, 2025, p. 1-6 (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-15-td3.pdf))

Next: Innovation 2 (Delayed Policy Updates) and Innovation 3 (Target Policy Smoothing) further stabilize TD3 learning.

TD3 Innovations 2 & 3: Target Policy Smoothing and Delayed Updates

Target Policy Smoothing

Clipped Gaussian noise is added to target actions during critic updates:

$$a' = \text{clip}(\pi_{\theta'}(s') + \text{clip}(\epsilon, -c, c), a_{\min}, a_{\max}), \quad \epsilon \sim \mathcal{N}(0, \sigma^2)$$

This regularizes Q-value estimates and prevents the policy from exploiting critic errors.

(COS435/ ECE 433: Introduction to RL - Lecture 15: Advanced Actor Critic Methods, Ben Eysenbach, 2025, p. 1-6, search for "clipped_noise = (torch.randn_like" (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-15-td3.pdf))

Delayed Policy Updates

The actor is updated less frequently than critics (typically every 2 iterations), allowing critics to converge to more accurate estimates before updating the policy (similar to two-timescale learning discussed in stochastic actor-critic).

(COS435/ ECE 433: Introduction to RL - Lecture 15: Advanced Actor Critic Methods, Ben Eysenbach, 2025, p. 1-6, search for "policy_frequency: int = 2" (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-15-td3.pdf))

TD3 Algorithm Pseudocode

Algorithm: Twin Delayed DDPG (TD3)

Input:

α_θ (actor learning rate), α_ϕ (critic learning rate), γ (discount factor)

σ^2 (exploration noise variance), τ (soft update rate)

N (replay buffer capacity), B (minibatch size)

c (noise clip range), σ_{policy} (target policy noise), d (policy update delay)

Output: $\pi_\theta(s), Q_{\phi_1}(s,a), Q_{\phi_2}(s,a)$

---

1. Initialize actor π_θ, critics Q_{ϕ_1}, Q_{ϕ_2}

2. Initialize target networks: $\pi_{\theta'} \leftarrow \pi_\theta,\ Q_{\phi_1'} \leftarrow Q_{\phi_1},\ Q_{\phi_2'} \leftarrow Q_{\phi_2}$

3. Initialize replay buffer $\mathcal{D}$ with capacity N

4. For episode $= 1, 2, \ldots$:

  5. Observe initial state s_0

  6. For $t = 0, 1, 2, \ldots$:

    7. Select action with noise: $a_t = \pi_\theta(s_t) + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, \sigma^2)$

    8. Execute a_t, observe r_t, s_{t+1}

    9. Store transition (s_t, a_t, r_t, s_{t+1}) in $\mathcal{D}$

    10. Sample random minibatch of B transitions from $\mathcal{D}$

    11. For each transition in minibatch:

      $\tilde{a}_i = \pi_{\theta'}(s_{i+1}) + \text{clip}(\epsilon, -c, c), \quad \epsilon \sim \mathcal{N}(0, \sigma^2_{\text{policy}})$

      Compute TD target: $y_i = r_i + \gamma \min_{j=1,2} Q_{\phi_j'}(s_{i+1}, \tilde{a}_i)$

    12. Update critics: $\phi_j \leftarrow \phi_j - \alpha_\phi \frac{1}{B} \sum_i \nabla_{\phi_j} (Q_{\phi_j}(s_i, a_i) - y_i)^2, \quad j = 1, 2$

    13. If $t \bmod d = 0$:

      14. Update actor: $\theta \leftarrow \theta + \alpha_\theta \frac{1}{B} \sum_i \nabla_a Q_{\phi_1}(s_i, a)|_{a=\pi_\theta(s_i)} \nabla_\theta \pi_\theta(s_i)$

      15. Soft update target networks:

        $\theta' \leftarrow \tau\theta + (1-\tau)\theta'$

        $\phi_j' \leftarrow \tau\phi_j + (1-\tau)\phi_j', \quad j = 1, 2$

    16. If terminal, break

17. Return $\pi_\theta(s), Q_{\phi_1}(s,a), Q_{\phi_2}(s,a)$

Key Changes from DDPG (highlighted in blue):

- **Two critics** Q_{ϕ_1} and Q_{ϕ_2} with separate target networks
- **Target policy smoothing:** Add clipped noise $\text{clip}(\epsilon, -c, c)$ to target actions
- **Min operation:** Use $\min(Q_{\phi_1'}, Q_{\phi_2'})$ for TD target to reduce overestimation
- **Delayed updates:** Actor and target networks updated every d steps (not every step)

In [5]:
```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import importlib
import actor_critic_helpers
importlib.reload(actor_critic_helpers)
from actor_critic_helpers import (plot_training_progress, render_
episode,
                                  Actor, Critic, select_action_w
ith_noise, ReplayBuffer,
                                  soft_update_target_networks, u
pdate_actor_batch)


def compute_td_target_batch_td3(critic1_target, critic2_target, a
ctor_target,
                                rewards, next_states, dones, gam
ma=0.99,
                                policy_noise=0.2, noise_clip=0.
5, action_low=-1, action_high=1):
    """
    Compute TD targets for TD3 using min of two target critics wi
th target policy smoothing.

    Args:
        critic1_target: First target critic network
        critic2_target: Second target critic network
        actor_target: Target actor network
        rewards: Batch of rewards
        next_states: Batch of next states
        dones: Batch of done flags
        gamma: Discount factor
        policy_noise: Std of noise added to target actions
        noise_clip: Clip range for noise
        action_low: Minimum action value
        action_high: Maximum action value

    Returns:
        TD targets as tensor
    """
    with torch.no_grad():
        next_states_tensor = torch.FloatTensor(next_states)

        # Get target actions
        next_actions = actor_target(next_states_tensor)

        # Add clipped noise to target actions (target policy smoo
thing)
        noise = torch.randn_like(next_actions) * policy_noise
        noise = torch.clamp(noise, -noise_clip, noise_clip)
        next_actions = torch.clamp(next_actions + noise, action_l
ow, action_high)

        # Compute Q-values from both target critics
        next_q1 = critic1_target(next_states_tensor, next_action
```

```
s).squeeze()
        next_q2 = critic2_target(next_states_tensor, next_action
s).squeeze()

        # Take minimum (reduces overestimation)
        next_q_values = torch.min(next_q1, next_q2)

        # Compute TD targets
        targets = torch.FloatTensor(rewards) + gamma * next_q_val
ues * (1 - torch.FloatTensor(dones))

    return targets


def update_critics_td3(critic1, critic2, critic1_optimizer, criti
c2_optimizer,
                       states, actions, targets):
    """
    Update both critics using batch MSE loss.

    Args:
        critic1: First critic network
        critic2: Second critic network
        critic1_optimizer: Optimizer for first critic
        critic2_optimizer: Optimizer for second critic
        states: Batch of states
        actions: Batch of actions
        targets: Batch of TD targets

    Returns:
        Tuple of (critic1_loss, critic2_loss)
    """
    states_tensor = torch.FloatTensor(states)
    actions_tensor = torch.FloatTensor(actions)

    # Compute Q-values from both critics
    q1_values = critic1(states_tensor, actions_tensor).squeeze()
    q2_values = critic2(states_tensor, actions_tensor).squeeze()

    # Compute MSE losses
    critic1_loss = nn.MSELoss()(q1_values, targets)
    critic2_loss = nn.MSELoss()(q2_values, targets)

    # Update critic 1
    critic1_optimizer.zero_grad()
    critic1_loss.backward()
    critic1_optimizer.step()

    # Update critic 2
    critic2_optimizer.zero_grad()
    critic2_loss.backward()
    critic2_optimizer.step()

    return critic1_loss.item(), critic2_loss.item()


def td3(env, state_dim, action_dim,
```

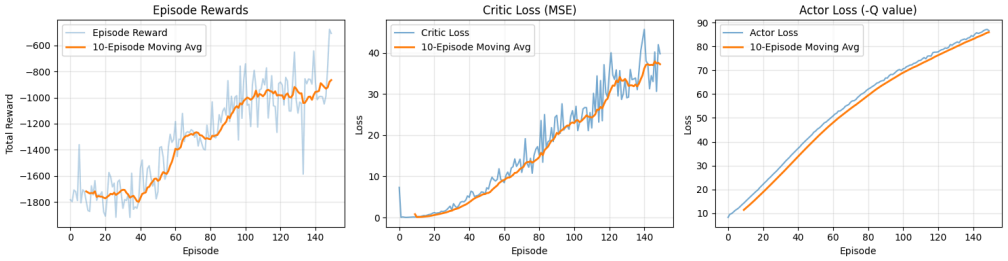
```
        num_episodes=1000, max_steps=1000,
        gamma=0.99, actor_lr=1e-4, critic_lr=1e-3,
        noise_std=0.1, tau=0.005, buffer_capacity=100000, batch_s
ize=64,
        policy_noise=0.2, noise_clip=0.5, policy_freq=2,
        plot_freq=10, render_env=None, render_freq=None):
    """
    Twin Delayed DDPG (TD3) Algorithm

    Implements the algorithm from the pseudocode with three key i
mprovements over DDPG:
    1. Two critics (Q_φ1, Q_φ2) with min operation to reduce over
estimation
    2. Target policy smoothing: add clipped noise to target actio
ns
    3. Delayed policy updates: update actor every d steps

    Args:
        env: Gym environment
        state_dim: Dimension of state space
        action_dim: Dimension of action space
        num_episodes: Number of training episodes
        max_steps: Maximum steps per episode
        gamma: Discount factor
        actor_lr: Learning rate for actor
        critic_lr: Learning rate for critics
        noise_std: Standard deviation of exploration noise
        tau: Soft update parameter for target networks
        buffer_capacity: Maximum capacity of replay buffer
        batch_size: Minibatch size for training
        policy_noise: Std of noise added to target actions
        noise_clip: Clip range for target action noise
        policy_freq: Frequency of delayed policy updates
        plot_freq: Frequency of plotting updates
        render_env: Optional environment for rendering during tra
ining
        render_freq: Frequency of rendering episodes

    Returns:
        Tuple of (actor, critic1, critic2, episode_rewards, actor
_losses, critic_losses)
    """
    # Step 1: Initialize networks
    actor = Actor(state_dim, action_dim)
    critic1 = Critic(state_dim, action_dim)
    critic2 = Critic(state_dim, action_dim)
    actor_optimizer = optim.Adam(actor.parameters(), lr=actor_lr)
    critic1_optimizer = optim.Adam(critic1.parameters(), lr=criti
c_lr)
    critic2_optimizer = optim.Adam(critic2.parameters(), lr=criti
c_lr)

    # Step 2: Initialize target networks
    actor_target = Actor(state_dim, action_dim)
    critic1_target = Critic(state_dim, action_dim)
    critic2_target = Critic(state_dim, action_dim)
    actor_target.load_state_dict(actor.state_dict())
```

```
    critic1_target.load_state_dict(critic1.state_dict())
    critic2_target.load_state_dict(critic2.state_dict())

    # Step 3: Initialize replay buffer
    replay_buffer = ReplayBuffer(buffer_capacity)

    episode_rewards = []
    episode_actor_losses = []
    episode_critic_losses = []

    total_steps = 0

    # Step 4: For each episode
    for episode in range(num_episodes):
        # Step 5: Reset environment
        state, info = env.reset()
        episode_reward = 0
        actor_losses = []
        critic_losses = []

        # Step 6: For each time step
        for t in range(max_steps):
            # Step 7: Select action with exploration noise
            action = select_action_with_noise(actor, state, noise
_std)

            # Step 8: Execute action and observe
            next_state, reward, terminated, truncated, info = en
v.step(action)
            done = terminated or truncated
            episode_reward += reward

            # Step 9: Store transition in replay buffer
            replay_buffer.push(state, action, reward, next_state,
done)

            # Step 10-16: Update networks if enough samples in bu
ffer
            if len(replay_buffer) >= batch_size:
                # Step 10: Sample random minibatch
                states, actions, rewards, next_states, dones = re
play_buffer.sample(batch_size)

                # Step 11-12: Compute TD targets and update criti
cs
                targets = compute_td_target_batch_td3(
                    critic1_target, critic2_target, actor_target,
                    rewards, next_states, dones, gamma, policy_no
ise, noise_clip
                )
                critic1_loss, critic2_loss = update_critics_td3(
                    critic1, critic2, critic1_optimizer, critic2_
optimizer,
                    states, actions, targets
                )
                critic_losses.append((critic1_loss + critic2_los
s) / 2)
```

```
                # Step 13-15: Delayed policy updates
                if total_steps % policy_freq == 0:
                    # Step 14: Update actor (reuse update_actor_b
atch from helpers)
                    actor_loss = update_actor_batch(actor, actor_
optimizer, critic1, states)
                    actor_losses.append(actor_loss)

                    # Step 15: Soft update target networks
                    soft_update_target_networks(actor, actor_targ
et, tau)
                    soft_update_target_networks(critic1, critic1_
target, tau)
                    soft_update_target_networks(critic2, critic2_
target, tau)

            total_steps += 1
            state = next_state

            # Step 16: Check if terminal
            if done:
                break

        # Store episode statistics
        episode_rewards.append(episode_reward)
        if len(actor_losses) > 0:
            episode_actor_losses.append(np.mean(actor_losses))
            episode_critic_losses.append(np.mean(critic_losses))
        else:
            episode_actor_losses.append(0)
            episode_critic_losses.append(0)

        # Plot progress
        if (episode + 1) % plot_freq == 0:
            plot_training_progress(episode_rewards, episode_actor
_losses,
                                   episode_critic_losses)

        # Render episode if requested
        if render_env is not None and render_freq is not None:
            if (episode + 1) % render_freq == 0:
                print(f"\nRendering episode {episode + 1}...")
                render_reward = render_episode(render_env, actor,
max_steps)
                print(f"Render episode reward: {render_reward:.2
f}")

    # Step 17: Return trained networks and statistics
    return actor, critic1, critic2, episode_rewards, episode_acto
r_losses, episode_critic_losses
```

In [6]:

```python
import gymnasium as gym
import numpy as np
from actor_critic_helpers import plot_rendered_episode

# Create two environments: one for training, one for rendering
env_train = gym.make('Pendulum-v1')
env_render = gym.make('Pendulum-v1', render_mode='human')

# Get environment dimensions
state_dim = env_train.observation_space.shape[0]
action_dim = env_train.action_space.shape[0]

print(f"State dimension: {state_dim}")
print(f"Action dimension: {action_dim}")

# Train TD3
actor, critic1, critic2, rewards_td3, actor_losses, critic_losses
= td3(
    env=env_train,
    state_dim=state_dim,
    action_dim=action_dim,
    num_episodes=150,
    max_steps=200,
    gamma=0.99,
    actor_lr=3e-4,
    critic_lr=1e-3,
    noise_std=0.3,
    tau=0.001,
    buffer_capacity=100000,
    batch_size=64,
    policy_noise=0.2,
    noise_clip=0.5,
    policy_freq=2,
    plot_freq=10,
    render_env=env_render,
    render_freq=50
)
print("\nTD3 Training completed!")
print(f"Final average reward (last 10 episodes): {np.mean(rewards
_td3[-10:]):.2f}")

# Close environments
env_train.close()
env_render.close()
```




```
Rendering episode 150...
Render episode reward: -623.87

TD3 Training completed!
Final average reward (last 10 episodes): -865.53
```

Why is TD3 Learning Slower than DDPG?

Comparison of Implementations:

Parameter	DDPG	TD3	Impact
Actor updates per step	Every step	Every 2nd step (`policy_freq=2`)	TD3 actor learns **50% slower**
Critic updates per step	1 critic	2 critics	TD3 has **2x critic computation**
Target computation	Single Q-value	Min of two Q-values + noise	TD3 has **more conservative targets**
Actor learning rate	`3e-4`	`3e-4`	Same
Tau (target update)	`0.001`	`0.001`	Same

Root Causes of Slower Learning:

1. **Delayed Policy Updates (`policy_freq=2`):**
   - TD3 updates actor every 2 steps vs DDPG every step
   - Result: Actor receives **half the gradient updates**
   - This is by design to reduce variance, but slows convergence
2. **Conservative TD Targets:**
   - $\min(Q_1, Q_2)$ underestimates Q-values
   - Lower targets → slower value propagation
   - Trade-off: More stable but slower
3. **Target Policy Smoothing:**
   - Adding noise to target actions introduces additional variance
   - Targets are less precise initially

Solutions to Make TD3 Faster:

Option 1: Reduce Delayed Update Frequency (easiest)

- Change `policy_freq=2` → `policy_freq=1` (update every step like DDPG)
- Pro: Matches DDPG update frequency
- Con: Loses some stability benefits of delayed updates

Option 2: Increase Actor Learning Rate (compensates for fewer updates)

- Change `actor_lr=3e-4` → `actor_lr=6e-4` (double it)
- Pro: Each actor update makes bigger progress
- Con: Risk of instability

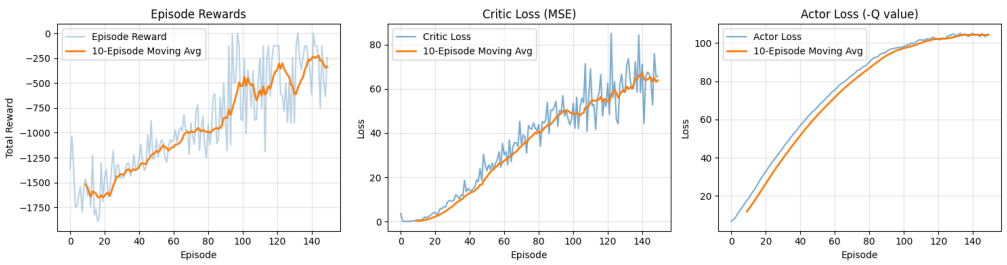
Option 3: Increase Target Update Rate (faster target network convergence)

- Change `tau=0.001` → `tau=0.005` (5x faster)
- Pro: Target networks track current networks faster

- Con: Can reduce stability

Let's test Option 1:

In [7]:
```
# Train TD3 with improved hyperparameters (Option 4: policy_freq=
1)
actor_fast, critic1_fast, critic2_fast, rewards_fast, actor_losse
s_fast, critic_losses_fast = td3(
    env=env_train,
    state_dim=state_dim,
    action_dim=action_dim,
    num_episodes=150,
    max_steps=200,
    gamma=0.99,
    actor_lr=3e-4,       # Same as DDPG
    critic_lr=1e-3,      # Same as DDPG
    noise_std=0.3,
    tau=0.001,           # Same as DDPG
    buffer_capacity=100000,
    batch_size=64,
    policy_noise=0.2,
    noise_clip=0.5,
    policy_freq=1,       # Changed from 2 → 1 (update every step l
ike DDPG)
    plot_freq=10,
    render_env=env_render,
    render_freq=200
)

print("\nTD3 with Faster Updates Training completed!")
print(f"Final average reward (last 10 episodes): {np.mean(rewards
_fast[-10:]):.2f}")
print(f"\nComparison:")
print(f"  DDPG final reward: {np.mean(rewards_ddpg[-10:]):.2f}")
print(f"  TD3 (slow) final reward: {np.mean(rewards_td3[-10:]):.2
f}")
print(f"  TD3 (fast) final reward: {np.mean(rewards_fast[-10:]):.
2f}")
```




```
TD3 with Faster Updates Training completed!
Final average reward (last 10 episodes): -336.22

Comparison:
  DDPG final reward: -551.66
  TD3 (slow) final reward: -865.53
  TD3 (fast) final reward: -336.22
```

Summary

Actor-Critic: Combines policy π_θ (actor) with Q-function Q_ϕ (critic)

Key Insight: Replace high-variance returns G_t with learned Q-values in policy gradient:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\left[\nabla_\theta \log \pi_\theta(a|s) \cdot Q^{\pi_\theta}(s,a)\right]$$

Methods:

- **Stochastic AC:** Q-based critic, discrete actions
- **DPG:** Deterministic policies for continuous control
- **OPDAC:** Deterministic Actor-Critic for continuous control
- **DDPG:** Deep Deterministic Policy Gradients (OPDAC + replay buffer + target networks)
- **TD3:** Double Q-learning + target smoothing + delayed updates

Convergence: Critic → Q^{π_θ}, Actor → local optimum (requires $\alpha_\phi >> \alpha_\theta$)

Remaining Gaps

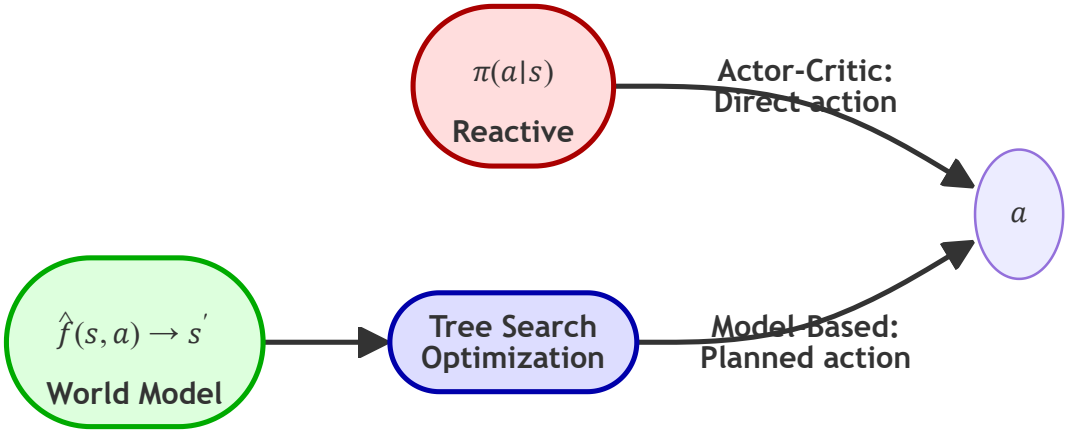
What Actor-Critic Doesn't Solve:

1. **Reactive Policies:** Actor-critic methods learn purely reactive policies $\pi(a|s)$ - no explicit planning or lookahead
2. **Sample Efficiency:** Still requires millions of environment interactions to learn
3. **No World Model:** Cannot predict future states or consequences of actions
4. **Wasted Experience:** Each transition used once for updates, then discarded
5. **Zero-Shot Generalization:** Cannot adapt to new goals without retraining

Outlook: Model-Based Reinforcement Learning

Beyond Reactive Policies: Instead of learning purely reactive mappings $\pi(a|s)$, learn a world model and **plan ahead**.

Core Idea: Learn dynamics model $\hat{f}(s,a) \approx s'$ and use it for planning:



(Sutton, Dyna Architecture, 1991 (http://incompleteideas.net/papers/sutton-91a.pdf);

Two Paradigms:

Key Challenge: Model errors compound during multi-step predictions - balancing model accuracy vs. computational cost.

1. Model-Predictive Control (MPC): Planning at Execution

- Learn dynamics model $\hat{f}$ offline or online- **Benefit:** Zero-shot adaptation to new reward functions without retraining policy
- At each timestep: Optimize action sequence using model lookahead- **Applications:** Robotics (manipulation, locomotion), autonomous driving
- Execute first action, replan at next state

2. Dyna-Q: Model-Augmented Learning

- Learn model from real experience: $\hat{f}(s,a) \to s', \hat{r}(s,a) \to r$- **Safety:** Test dangerous actions in simulation first
- Use model to generate **synthetic experience** for training- **Flexibility:** Same model + different rewards = different behaviors
- Update Q-function with both real and simulated transitions- **Reasoning:** Explicit lookahead vs. implicit reactive behavior
- **Benefit:** Reuse experience - each real transition enables multiple model-based updates- **Sample Efficiency:** Model enables unlimited simulated practice